Lab Handout 3-C

First Live Frame — Systematic Integration Debugging

Capstone — Module 3: System Integration and Data Collection

1 Objective

Connect the STM32 interceptor (Threat) to the Field-Programmable Gate Array (FPGA) and work through the four debugging layers (physical interface, baud rate and framing, protocol synchronization, data integrity) until the ILA captures the first `frame_valid` pulse on live hardware. This handout is the central milestone of Module 3: a validated end-to-end data path spanning all four hardware components (cooperative target Microcontroller Unit (MCU) → Radio Frequency (RF) link → interceptor MCU → FPGA fabric).

2 Prerequisites

- Lab Handout 3-A complete (all 9 checklist items passed, wiring plan documented).
- Lab Handout 3-B complete (debug bitstream programmed, ILA dashboard verified with 12 probes).

Make Connections Now

This is the first handout that physically connects boards together. Follow the wiring plan from LH3-A exactly. Common-ground first, then signal. Verify the voltage levels (3.3 V on both sides) before applying power.

3 Procedure

3.1 Step 1 — Physical connection

1. With all boards **powered off**, run a GND jumper wire from the STM32 Threat Nucleo GND header pin to the Nexys A7 PMOD JA pin 5 (or pin 11, both are GND). This is the most important connection.
2. Run a signal wire from the Threat's `USART3 TX` (`PC4`) to Nexys A7 `JA[0]` (Artix-7 ball `C17`). Verify the `.xdc` constraint from LH2-A/LH3-B names this port `uart_rxd`.
3. Power on the Nexys A7 first (so its Universal Asynchronous Receiver/Transmitter (UART) receiver is active before the STM32 starts transmitting).
4. Power on the Threat Nucleo. Then power on both Target Nucleos (Target A and Target B for ping-pong traffic).

[Figure placeholder]

`LH3C_pmod_wiring.jpg`

Threat STM32 `USART3 TX` (`PC4`) wired to Nexys A7 `JA[0]` with a dedicated GND jumper. The GND wire (black) must be present; without it, the UART signal levels are referenced to different potentials.

Figure 1: Threat STM32 `USART3 TX` (`PC4`) wired to Nexys A7 `JA[0]` with a dedicated GND jumper. The GND wire (black) must be present; without it, the UART signal levels are referenced to different potentials.

3.2 Step 2 — Layer 1: Physical interface check

1. In the Vivado ILA dashboard, arm the trigger on `uart_rx_pin` falling edge (start bit of any byte).
2. Wait up to 5 seconds. The Threat transmits a forwarding frame every time it intercepts a packet (DATA + ACK pairs at ∼1 s, so ∼2 frames per second; PAUSE bursts only when its user button is held).
3. **If the trigger fires:** the physical layer is working. Proceed to Step 3.
4. **If the trigger does not fire:** the signal is not reaching the FPGA. Debug:
   - Confirm the Threat's `USART3 TX` is on `PC4` (check the HAL initialization in the LH1-G firmware).
   - Confirm the `.xdc` constraint file maps `uart_rx_pin` to `C17` and is active (not commented out).
   - With a DMM in DC voltage mode, probe the signal wire at the PMOD connector while the STM32 is running. You should see a voltage between 1.5 V and 3.3 V (the DMM averages the UART idle/active transitions). A reading of exactly 0 V means no signal or a broken wire.

3.3 Step 3 — Layer 2: Baud rate and framing check

1. Arm the trigger on `rx_strobe` rising edge.
2. When the trigger fires, examine the `rx_data` waveform. The first byte of a forwarding frame should be `0x7E`. Check several captures to determine if bytes are decoding correctly or producing garbage.
3. Monitor `fifo_full`. If it asserts frequently, bytes are arriving faster than the parser can consume them — this indicates a baud rate mismatch causing runaway byte generation. Compute

the expected divisor: for a 100 MHz FPGA clock and 115200 baud, the divisor is 100,000,000/115,200 ≈ 868. Check the VHSIC Hardware Description Language (VHDL) `uart_rx` generic.

4. If `rx_data` consistently decodes as garbage values (not valid frame bytes), the STM32 may be configured for a different baud rate or 2 stop bits while the FPGA expects 1. Check the HAL UART initialization in the Threat firmware.

Baud Rate Deviation Tolerance

The Module 2 UART receiver uses 16× oversampling and samples at the bit midpoint, which provides ±3% baud rate tolerance. The STM32L476 derives its UART clock from the APB1 bus; if the PLL configuration in the Threat firmware is different from what was used in LH1-G testing, the baud rate may be off. If `rx_error` appears on every frame, measure the actual bit period with a logic analyzer or oscilloscope and compute the deviation:

$$\text{Deviation (\%)} = \frac{|T_{\text{measured}} - 8.68\,\mu\text{s}|}{8.68\,\mu\text{s}} \times 100$$

A deviation above 3% requires adjusting the STM32 APB1 prescaler or the FPGA baud divisor.

3.4 Step 4 — Layer 3: Protocol synchronization check

1. With `rx_strobe` triggering and bytes decoding correctly, the key observable is `frame_valid` and `frame_reject`. Set the ILA trigger on `rx_strobe` rising edge and capture 50 consecutive `rx_data` values. Scroll through the waveform and confirm `0x7E` appears as the first byte of each frame.

2. If `0x7E` never appears in any capture, the Threat firmware is transmitting a different frame format than expected by the parser. Open the Threat debug UART terminal simultaneously and watch one complete packet reception. The forwarding frame format (from Module 1 Table 6) should match what is in the ILA capture. If the Threat firmware was modified since LH1-G, it may have changed the start delimiter or frame structure.

3. To confirm the full pipeline is working, change the trigger to `frame_valid` rising edge and click **Run Trigger**. Arm and wait — when it fires, `frame_valid` will show a pulse in the waveform and `frame_reject` should remain at 0. If `frame_valid` never fires, scroll left in the waveform from the last `rx_strobe` capture and check that the byte immediately after `0x7E` is `0x29` (the length byte for a 43-byte frame). If a different value appears at offset 1, the parser will reject the frame. Compare the captured byte stream against the forwarding frame definition in LH1-G.

[Figure placeholder]

`LH3C_parser_fsm_capture.png`

ILA capture showing `rx_data` bytes for a live frame. The `0x7E` start delimiter is visible at the beginning of the sequence, followed by the `0x29` length byte. `frame_valid` asserts after the final byte is processed.

Figure 2: ILA capture showing `rx_data` bytes for a live frame. The `0x7E` start delimiter is visible at the beginning of the sequence, followed by the `0x29` length byte. `frame_valid` asserts after the final byte is processed.

3.5 Step 5 — Layer 4: Data integrity (CRC) check

1. If the parser reaches `CRC_CHECK` but then asserts `frame_reject` instead of `frame_valid`, the CRC is failing.

2. Set the ILA trigger on `frame_reject` rising edge. Capture the complete 43-byte frame using ILA pre-trigger (set pre-trigger to 45 samples so the full frame precedes the reject event).

3. Export the captured bytes from the ILA waveform viewer (**File** → **Export** → **ILA Data**) and manually compute CRC-8 over bytes 1–41 in Python:

```
def crc8(data, poly=0x07, init=0x00):
    crc = init
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = ((crc << 1) ^ poly) & 0xFF if (crc & 0x80) else (crc << 1) & 0xFF
    return crc

# Replace with your captured bytes:
frame_bytes = [0x01, 0x07, 0x0E, ...]   # bytes 1..41 of the forwarding frame
computed = crc8(frame_bytes)
captured_crc = 0xXX  # byte 42 from the ILA capture
print(f"Computed:  {computed:#04x}")
print(f"Captured:  {captured_crc:#04x}")
print(f"Match:     {computed == captured_crc}")
```

4. If the computed CRC matches the captured byte 42 but the FPGA still rejects, the VHDL CRC engine has a bug (wrong polynomial, wrong initialization, wrong byte range). Re-run the Module 2 LH2-E testbench against this live frame as stimulus.

5. If the computed CRC does *not* match byte 42, the STM32 Threat firmware is computing CRC differently (different polynomial `0x31` vs `0x07`, different initialization value, or including/excluding the start delimiter). Compare against the Threat source from LH1-G.

CRC Polynomial Mismatch is the Most Common Layer-4 Failure

The STM32L476 hardware CRC peripheral defaults to polynomial `0x04C11DB7` (CRC-32), not CRC-8. If the LH1-G firmware uses the hardware CRC unit without reconfiguring it, or uses a software CRC-8 with a different polynomial than `0x07`, the result will never match the Module 2 VHDL CRC engine. Resolve by aligning both sides on polynomial `0x07`, initialization `0x00`, input/output no-reflect, covering bytes 1–41.

3.6 Step 6 — First frame milestone

1. Once all four layers are verified, arm the ILA trigger on `frame_valid` rising edge.
2. Before reading values, set `rssi_out` to display as a signed decimal so it can be compared directly to the Threat's debug log. In the waveform viewer, right-click the `rssi_out` row, select **Radix → Signed Decimal**. The value should appear as a negative number (e.g. −45) matching the `rssi_dbm` field in the Threat's UART output.
3. When the trigger fires, examine the captured values for `beacon_id`, `pkt_type`, and `rssi_out`.
4. Simultaneously read the Threat's debug UART terminal. The last reception line should show the same packet.
5. Confirm the following match between the ILA capture and the STM32 debug log:
   - `beacon_id` equals the originating station's configured ID (`0x01` for Target A DATA, `0x02` for Target B ACK).
   - `pkt_type` matches the expected packet type (`00`=DATA, `01`=ACK, `10`=PAUSE).
   - `rssi_out` is within ±2 dB of the Received Signal Strength Indicator (RSSI) printed by the Threat.
   - Both CRC flags pass: `frame_valid` = 1 and `frame_reject` = 0.
6. Optionally, hold the Threat's user button for 2 s and re-arm the trigger. The Threat continues forwarding any DATA and ACK frames it receives during the button hold, so `frame_valid` captures will still appear. Confirm the Threat's debug UART shows `PAUSE -> ID=...` lines simultaneously, and that the targeted Target's UART prints `"I'm Jammed"` — this validates the jam path without requiring a separate frame type at the FPGA.

[Figure placeholder]

`LH3C_first_frame_valid.png`

ILA capture of the first `frame_valid` pulse on live hardware. The `beacon_id` (Beacon ID) and `rssi_out` (RSSI) fields are highlighted for comparison against the STM32 receiver debug log.

Figure 3: ILA capture of the first `frame_valid` pulse on live hardware. The `beacon_id` (Beacon ID) and `rssi_out` (RSSI) fields are highlighted for comparison against the STM32 receiver debug log.

3.7 Step 7 — Ten-frame field comparison

1. Arm the ILA on `frame_valid` and capture 10 consecutive frames covering both DATA (Beacon ID `0x01`) and ACK (Beacon ID `0x02`) ping-pong rows. For each, record `beacon_id` (Beacon ID), `pkt_type` (packet type), and `rssi_out` (RSSI) from the ILA waveform.
2. In the Threat terminal, capture the corresponding 10 reception log lines.
3. Build a side-by-side comparison table:

Frame #	FPGA ID	FPGA pkt_type	Threat ID/type	FPGA RSSI	Threat RSSI
1					
2					
⋮					
10					

4. All Beacon IDs and `pkt_type` values should match exactly. RSSI discrepancies up to ±3 dB are acceptable (rounding in the 1-byte unsigned representation). Larger discrepancies indicate a field offset error in the parser or a byte-ordering issue in the RSSI extraction.

4 Verification Checklist

- ☐ Physical layer: `uart_rx_pin` transitions observed on ILA within 5 seconds of receiver power-on.
- ☐ Baud/framing: `rx_data` decodes correct hex values; `fifo_full` does not assert during normal traffic.
- ☐ Protocol sync: `0x7E` delimiter and `0x29` length byte confirmed in `rx_data` capture; `frame_valid` eventually asserts.
- ☐ Data integrity: `frame_valid` fires; `frame_reject` stays low on back-to-back frames.
- ☐ Ten-frame comparison: all `beacon_id` and `pkt_type` fields match Threat log; RSSI within ±3 dB.

Lab Handout 3-C Deliverable — Module 3 Integration Milestone

1. Integration debug log: for each debugging layer, document the symptoms observed, the diagnostic steps taken, and the root cause and resolution. If a layer had no issues, document that explicitly.
2. ILA waveform captures for: (a) the first `frame_valid` event, (b) the parser state transitions during a live frame, and (c) at least one diagnosed failure *before* the fix was applied (e.g., the `frame_reject` capture used for CRC analysis).
3. The 10-frame side-by-side comparison table (filled in).
4. Annotated photos of the physical inter-board wiring as it was when the first live frame was captured.

Glossary

FPGA	Field-Programmable Gate Array	1, 2, 3, 5, 6
MCU	Microcontroller Unit	1
RF	Radio Frequency	1
RSSI	Received Signal Strength Indicator	6, 7
UART	Universal Asynchronous Receiver/Transmitter	1, 2, 3, 4, 6
VHDL	VHSIC Hardware Description Language	3, 5